

Boosting

QwanxingxuA

2026 年 8 月 18 日

1 简介

Boosting 是机器学习中性能很好的方法。从方法应用的角度来看, 它解决了两个核心问题: 将模型注意力放到分类错误样本上; 组合多个模型, 并且不是平均个组件的功效, 而是通过“学习”这一强大的手段组合各个组件。

Boosting 是对决策树的强大改良, 在决策树那一章中笔者提到与神经网络的对比, 发现决策树一个难点就是无法像神经网络那样反向传播。但是 Boosting (提升树) 是可以类似的, 只是提升树采用“梯度上升”的方式。那么为什么决策树和提升树会出现不同呢? 笔者将在后续解释。

本文着重探讨 Boosting 的机器学习逻辑, 以数学辅以证明, 关于算法的详细步骤省略, 建议初次接触的读者参考李航老师书的算法部分。

本文主要针对分类模型, 会涉及回归模型。下面给出分类问题的前提定义。

定义样本空间 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, 输入空间 $\mathcal{X}$, 输出空间 $\mathcal{Y} = \{-1, +1\}$ 。

2 加法模型, AdaBoost

2.1 加法模型

加法模型顾名思义就是加在一起的模型, 即

$$f(x) = \sum_{m=1}^M \alpha_m b(x; \theta_m)$$

其中 b 是基函数, θ_m 是基函数的参数。

2.2 AdaBoost

AdaBoost 本质依旧是加法模型，只在策略的选择上使用了指数的特例，并且其收敛性具有随机的特性。故而取名”AdaBoost”。

定义模型：

$$f(x) = \sum_{m=1}^M \alpha_m G_m(x)$$

对于当下特点的二分类任务，更严谨的定义是：

$$F(x) = \text{sign}(f(x))$$

$$f(x) = \sum_{m=1}^M \alpha_m G_m(x)$$

Boosting 算法采用递归提升的方法，有点类似于递归树的提升部分。用数学表达：

$$f_m(x) = f_{m-1}(x) + \alpha_m G_m(x)$$

下面考虑策略，AdaBoost 使用指数损失函数：

$$L(y, f(x)) = \exp(-yf(x))$$

于是可以构建 AdaBoost 的优化问题：

$$\arg \min_{\alpha_m, G_m} \sum_{i=1}^n \exp[-y_i(f_{m-1}(x) + \alpha_m G_m(x))]$$

令 $w_{mi} = \exp(-y_i f_{m-1}(x))$

$$\arg \min_{\alpha_m, G_m} \sum_{i=1}^n w_{mi} \exp(-y_i G_m(x)) = \sum_{y_i = G_m(x_i)} w_{mi} e^{-\alpha_m} + \sum_{y_i \neq G_m(x_i)} w_{mi} e^{\alpha_m}$$

下面求解上述问题。对于该分类任务，策略是要减少分类错误的概率。故而得到子问题，即 G_m 的解

$$G_m^* = \arg \min_{G_m} \sum_{i=1}^n w_{mi} I(G_m(x) \neq y_i)$$

这里容易误解，其实就是原始问题的第二部分的等价优化问题。优化问题转换成等价问题是很常见的手段。

令 $e_m = \sum_{i=1}^n w_{mi} I(G_m^*(x) \neq y_i)$, 对目标函数求导

$$\frac{\partial L}{\partial \alpha_m} = -e^{-\alpha_m}(1 - e_m) + e^{\alpha} e_m = 0$$

得到

$$\alpha_m^* = \frac{1}{2} \log \frac{1 - e_m}{e_m}$$

以上便是 AdaBoost 的算法过程, 其实 Adaboost 本身就是模型、策略、算法, 只是平常我们更多是使用的算法。笔者为了迎合这种现象, 现将上述求解的结果整理成下面的算法组件。

$$G_m = \arg \min e_m$$

$$e_m = \sum_{i=1}^n w_{mi} I(G_m(x) \neq y_i)$$

$$\alpha_m = \frac{1}{2} \log \frac{1 - e_m}{e_m}$$

$$w_{mi} = \exp(-y_i f_{m-1}(x_i)) = \frac{w_{m-1,i}}{Z_m} \exp(-y_i \alpha_{m-1} G_{m-1}(x_i))$$

$$Z_m = \sum_{i=1}^n \exp(-y_i \alpha_{m-1} G_{m-1}(x_i))$$

其中 Z_m 是规范化因子。

AdaBoost 训练误差上界

AdaBoost 训练误差上界为:

$$\frac{1}{n} \sum_{i=1}^n I(F(x_i) \neq y_i) \leq \frac{1}{n} \sum_{i=1}^n \exp(-y_i f(x_i)) = \prod_{m=1}^M Z_m$$

证明:

1. 先证明前半部分:

对于分类错误样本有 $-y_i f(x_i) \geq 0$, 则

$$e^{-y_i f(x_i)} \geq 1$$

$$\frac{1}{n} \sum_{i=1}^n I(F(x_i) \neq y_i) \leq \frac{1}{n} \sum_{i=1}^n \exp(-y_i f(x_i))$$

证毕

2. 再证明后半部分:

$$\begin{aligned}
\frac{1}{n} \sum_{i=1}^n \exp(-y_i f(x_i)) &= \frac{1}{n} \sum_{i=1}^n \exp(-y_i \sum_{m=1}^M \alpha_m G_m(x_i)) \\
&= \frac{1}{n} \sum_{i=1}^n \prod_{m=1}^M \exp(-y_i \alpha_m G_m(x_i)) \\
&= \frac{1}{n} \sum_{i=1}^n \frac{w_{M+1}}{w_1} \prod_{m=1}^M Z_m
\end{aligned}$$

由于 $w_1 = \frac{1}{n}, \sum_{i=1}^n w_{M+1} = 1$

$$\frac{1}{n} \sum_{i=1}^n \exp(-y_i f(x_i)) = \prod_{m=1}^M Z_m$$

证毕

针对二分类问题, 上式还可以扩展:

$$\begin{aligned}
Z_m &= \sum_{y_i=G_m(x_i)} w_{mi} e^{-\alpha_m} + \sum_{y_i \neq G_m(x_i)} w_{mi} e^{\alpha_m} \\
&= (1 - e_m) e^{-\alpha_m} + e_m e^{\alpha_m} \\
&= 2\sqrt{e_m(1 - e_m)}
\end{aligned}$$

令 $\gamma_m = \frac{1}{2} - e_m$, 由于 $e^{-x} \geq \sqrt{1 - 2x}$

$$Z_m = \sqrt{1 - 4\gamma_m^2} \leq e^{-2\gamma_m^2}$$

所以

$$\prod_{m=1}^M Z_m \leq \exp(-2 \sum_{m=1}^M \gamma_m^2)$$

如果存在 $\gamma > 0$ 且对所有的 m 均有 $\gamma_m \geq \gamma$, 则

$$\prod_{m=1}^M Z_m \leq e^{-2M\gamma^2}$$

这个表达式说明了算法呈指数收敛, 这是说明算法具有非常好的性能。
由于 γ 具有随机性, 所以算法取名 Adaboost。

3 提升树

3.1 提升树

提升树是二叉树，本质也是加法模型。与 Adaboost 在策略上具有差异，但是在平方误差下和 CART 算法类似。

假设提升树得到 J 个叶子节点 $\{R_1, R_2, \dots, R_J\}$, 定义提升树模型：

$$f(x) = \sum_{m=1}^M T(x; \Theta_m)$$

$$f_m(x) = f_{m-1}(x) + T(x; \Theta_m)$$

$$T(x; \Theta_m) = \sum_{j=1}^J c_j I(x \in R_j)$$

这个定义和 CART 回归树是类似的。

取平方误差损失函数：

$$L(y, f_m(x)) = (y - f_{m-1}(x) - T(x; \Theta_m))^2 = (r - T(x; \Theta_m))^2$$

其中， r 是上一子树的残差。

提升树是二叉树，定义

$$R_1 = \{x \mid x \leq s\}, R_2 = \{x \mid x > s\}$$

则问题转化为

$$\min_s \min_{c_1, c_2} [(r_1 - c_1)^2 + (r_2 - c_2)^2]$$

这个问题也同 CART 算法类似，具体不再赘述。

当取一些其它损失函数的时候很难之间求解问题，但是提升树的好处就在于是可以求导传递的，只不过是以梯度提升的方法。这也说明树也可以构建微分网络。

3.2 提升树与决策树

就上面提升树回归树例子来看，提升树与决策树 CART 简直一样，但实际上这是不对的。

CART 回归树如笔者在决策树说的，它是单层模型，虽然各层次之间存在联系，但是模型本身是忽略了这种联系的。严格的说，决策树基于单层次的极大似然或者说最小条件熵构建，本质依旧只是单层模型。提升树的加法模型是整体的，这种层次间的模型是严格出现在模型表达式上面的。就 CART 回归树来看，它是基于标签与叶子节点的损失，但是提升树是残差与叶子节点的损失。这是二者在平方损失误差下本质的区别，残差记录了各层次之间的联系，但是 CART 只是浅显的受使用过的特征联系而已。

提升树是决策树的强大改良。笔者在决策树那一章提到决策树的神经网络构想，其实现在想来是不成立的，因为决策树的网络联系是浅显的，这是模型本身决定的，难以传播也是必然的。提升树是有可能的，因为加法模型本身就具体以上特性。

实际上 CART 剪枝是比较生硬的，它没有神经网络那样基于交叉熵损失函数反向传播优化的便利。同时，它也没有提升树这样基于损失函数逐步提升的“传播”。笔者在决策树那一章说剪枝如图神经网络的 softmax with crossentropyloss 层，现在看来是完全达不到的。